Pontificia Universidad Católica Argentina
Facultad de Ingeniería y Ciencias Agrarias

Estadística Aplicada y Modelos de Regresión (EAyMR 2026)
Trabajo Práctico Integrador: Decision Intelligence Capstone

¿El top-5 vende, o solo muestra lo que se vendería igual?

Retorno causal de un algoritmo de recomendación de hoteles

Verónica López Marsico (Project Lead & Scrum Master)
Lucía Báez (Backend & ML / Causal Engineer)
Antonella Robiollio (Frontend & UI/UX + QA & Validation Engineer)

18 de noviembre de 2026

Profesor titular: Prof. Ivan Gonzalo Vilaboa

Licenciatura en Ciencia de Datos / Ingeniería en Inteligencia Artificial
2.º año, 2.º semestre

Estimamos el efecto causal de que el recomendador de Expedia exponga un hotel en los cinco primeros lugares de una búsqueda sobre su probabilidad de reserva, con los datos del desafío ICDM 2013. Un subconjunto de búsquedas mostró los hoteles en orden aleatorio: lo usamos como benchmark cuasi-experimental para contrastar empíricamente el supuesto Backdoor. El trabajo integra inferencia clásica, diagnóstico de Gauss-Markov y PCA, un torneo predictivo con calibración y la estimación causal con DoWhy, logit condicional por búsqueda y Double Machine Learning. Todo número del texto se lee de `models/resultados.json`, que genera el notebook anexo (`notebooks/TP_Integrador_Final.ipynb`); el código está en el repositorio https://github.com/verolopezmar/TPI_Estadistica-avanzada-.

Tabla de contenidos

Resumen ejecutivo 3

1 Introducción y preguntas de negocio 3

2 Marco formal 4
- 2.1 Unidades, variables y roles . . . 4
- 2.2 Estimandos . . . 4
- 2.3 Supuestos de identificación . . . 5
- 2.4 Diseño muestral y plan experimental . . . 5

3 Datos y auditoría de calidad 5

4 Fase 1: inferencia clásica y ANOVA 6

5 Fase 2: Gauss-Markov y PCA 7

6 Fase 3: torneo predictivo 9

7 Fase 4: inferencia causal 11
- 7.1 DAG e identificación . . . 11
- 7.2 DoWhy en cuatro pasos . . . 11
- 7.3 Positividad y overlap . . . 12
- 7.4 Escala relativa: observacional vs. cuasi-experimento . . . 12
- 7.5 Robustez de la conclusión central . . . 13
- 7.6 Mediador: el clic como paso del embudo . . . 14
- 7.7 Trampas de selección y del colisionador . . . 14
- 7.8 Refutaciones . . . 15
- 7.9 Double Machine Learning y CATE . . . 15

8 Ensayo crítico: capacidad predictiva vs. palanca causal 17

9 Recomendaciones de negocio cuantificadas 17

10 Limitaciones y amenazas a la validez 18

11 Conclusiones 19

Referencias 19

12 Anexo A: uso de inteligencia artificial generativa 22

13 Anexo B: reproducibilidad y Quality Gate 27

Listado de Figuras

1 Sesgo de posición: tasa de clic por posición con orden del algoritmo y aleatorio (IC 95 %), y precio relativo por categoría. . . . 7
2 ANOVA del log precio por estrellas: distribuciones, Q-Q por grupo y medias con IC 95 %. 7
3 Diagnóstico de residuos: residuos vs. ajustados, Q-Q y dispersión por cuantil de $\hat{y}$. . . . 8
4 Scree plot, varianza explicada acumulada y cargas de PC1 y PC2 (ajustado en Train). . . . 8
5 Curvas ROC y PR y diagramas de confiabilidad en Test. . . . 10

6 Importancia por MDI (Train) vs. permutación (Test). 10
7 DAG con el orden del algoritmo (izquierda) y con orden aleatorio (derecha, cirugía de grafo de do(T)). 11
8 Overlap de la propensión por grupo en cada estrato (zonas recortadas sombreadas) y estimadores del ATE con IC 95 % (orden del algoritmo). 12
9 OR intra-búsqueda: observacional (orden del algoritmo) vs. benchmark (orden aleatorio). 13
10 Robustez: balance antes y después de re-ponderar y distribución bootstrap de los cocientes. 14
11 CATE del top-5 (Causal Forest DML, orden del algoritmo) según estrellas, precio relativo, ubicación y usuario recurrente, con bandas de IC 95 %. 16
12 CATE del top-5 por segmento de hotel (terciles de precio relativo y de ubicación), con IC 95 %: orden aleatorio (LinearDML) y orden del algoritmo (Causal Forest). 16

Listado de Tablas

1 Variables y roles en el DAG. 4
2 Centinelas (0 = sin dato): distorsión antes y después de pasarlos a faltante. 5
3 Faltantes: porcentaje, mayor dependencia con variables observadas y tratamiento (medianas ajustadas solo con Train). 6
4 Auditoría de Gauss-Markov del MCO de referencia. 8
5 Desempeño en Test (umbral elegido en Train por máximo F1). 9
6 Odds ratios de la logística sin penalizar (Train, errores por clúster de búsqueda): las 12 variables con menor p-valor. 9
7 Efecto del top-5 sobre P(reserva) en escala absoluta (DoWhy) y sesgo de confusión del ingenuo. 11
8 Diagnóstico de positividad: propensión estimada $\hat{e}(z)$ por grupo, fracción fuera de [0,01; 0,99], peso máximo y tamaño efectivo de Kish. 12
9 OR intra-búsqueda del top-5 (logit condicional, ecuación 2) y diferencia de probabilidad con efectos fijos (MPL-EF). 12
10 Bootstrap por búsqueda (IC 95 % percentil): OR, efectos marginales (AME) y diferencia de riesgos con efectos fijos (DR-EF). 14
11 Trampas demostradas: ajustar por el mediador, condicionar en la selección S y en el colisionador C. 15
12 Refutaciones de DoWhy sobre la regresión backdoor (orden del algoritmo). 15
13 Heterogeneidad con orden aleatorio: LinearDML, $\tau(x) = \theta_0 + x^T\theta$ (R = 1). 16
14 Valuación del slot por escenarios (por 1.000 exposiciones en el top-5; búsquedas con al menos un clic). 17
15 Efecto causal por segmento con orden aleatorio (LinearDML, R = 1): reservas y USD incrementales por 1.000 exposiciones (gasto mediano, sin desplazamiento). 17

Resumen ejecutivo

- **El top-5 causa reservas, pero bastante menos de lo que sugieren los datos del algoritmo.** Con orden aleatorio, estar en el top-5 multiplica por 2,11 las chances de que un hotel sea el reservado de su lista (OR intra-búsqueda ajustado, IC 95 % [1,90; 2,35]). Con el orden del algoritmo, la comparación ingenua da 5,38 y el ajuste por los confusores observados la baja a 3,38.
- **El ajuste por lo observable no alcanza.** Explica el 50 % de la brecha en log-odds. El OR observacional ajustado supera al benchmark en un factor 1,60 (IC 95 % bootstrap por búsqueda [1,42;

1,80], 1000 réplicas); en diferencia de riesgos, que es colapsable, el factor es 1,65. El ranker usa información que no observamos.

- **Predecir no es intervenir.** El mejor modelo predictivo (Random Forest, PR-AUC 0,153 con una prevalencia del 2 %) usa la posición como variable principal, pero buena parte de ese poder predictivo es selección del algoritmo.
- **Para quién rinde el slot.** Con orden aleatorio, el efecto por exposición es 11,0 reservas cada 1.000 exposiciones para hoteles baratos y bien ubicados, contra 4,2 para hoteles caros y mal ubicados. Reasignar 1.000 exposiciones del segundo grupo al primero suma 6,9 reservas (≈ 1.530 USD con el gasto mediano).
- **Decisión.** Valuar el slot premium con el efecto causal, no con la tasa observada (el dato del algoritmo sobrevalúa el OR en 60 %), priorizar la exposición donde el CATE es mayor y mantener una fracción de tráfico con orden aleatorio como auditoría permanente.

1. Introducción y preguntas de negocio

Un buscador de hoteles muestra alrededor de 25 resultados por búsqueda, pero la atención del usuario se concentra en los primeros. Los slots superiores son un recurso escaso de suma cero: subir un hotel baja a otro. El recomendador decide quién los ocupa y, a la vez, suele elegir hoteles que se venderían igual (mejor ubicados, más baratos para la búsqueda, con buena reputación). La pregunta operativa es si el slot **causa** ventas o solo las **predice**: de eso depende cuánto vale un slot premium (publicidad, comisiones de posicionamiento) y a quién conviene asignarlo.

Preguntas de valor.

1. ¿Cuál es el efecto causal de la exposición en el top-5 sobre la reserva (ATE y CATE)?
2. ¿Cuánto sesgo introduce estimarlo con el orden del algoritmo en lugar del orden aleatorio?
3. ¿El efecto pasa entero por el clic?
4. ¿Para qué hoteles rinde más el slot?
5. *(Propias)* ¿Qué tan aleatorio es el orden aleatorio? ¿Cómo condiciona la selección del dataset lo que se puede estimar?

Restricciones reales del sistema. Los datos son confidenciales según las reglas de la competencia (Expedia y Kaggle 2013): ningún archivo de datos está en el repositorio y la aplicación usa solo agregados. El dataset incluye solo búsquedas con al menos un clic y, en el estrato del algoritmo, sobremuestrea las compras (sección 3).

2. Marco formal

Esta sección fija la notación, los estimandos y los supuestos **antes** de cualquier resultado. Distinguimos parámetros poblacionales (letras griegas: β, μ, τ, ψ) de sus estimadores ($\hat{\beta}, \bar{x}, \hat{\tau}, \hat{\psi}$).

2.1. Unidades, variables y roles

La unidad de análisis es el par búsqueda × hotel (s, i); la unidad independiente para la inferencia es la búsqueda s, porque los hoteles de una lista comparten usuario, fecha y destino.

Tabla 1: Variables y roles en el DAG.

Símbolo	Variable	Rol
Y_{is}	`booking_bool`	Resultado: el hotel i es el reservado en la búsqueda s
T_{is}	$1\{\texttt{position} \leq 5\}$	Tratamiento: exposición en el top-5 de la pantalla
M_{is}	`click_bool`	Mediador (y colisionador con la intención I)
Z	calidad y precio (Z_h), popularidad (Z_p), contexto (Z_b), usuario (Z_u), competidores (Z_c)	Confusores observados
U, I	desempeño histórico del hotel en Expedia; intención de la sesión	Latentes
R_s	`random_bool`	Mecanismo de asignación: 1 = orden aleatorio
S_s	inclusión de la búsqueda en el dataset	Selección (descendiente de M y, con $R = 0$, de Y)
C	"destacado en el reporte", simulado	Colisionador $T \to C \leftarrow Y$ para la trampa de Berkson

2.2. Estimandos

Con resultados potenciales $Y_{is}(t)$, $t \in \{0, 1\}$:

$$\tau = \mathbb{E}\big[Y(1) - Y(0)\big], \qquad \tau(x) = \mathbb{E}\big[Y(1) - Y(0) \mid X = x\big], \qquad \tau_S = \mathbb{E}\big[\tau(X) \mid X \in S\big]. \tag{1}$$

Como el dataset está seleccionado sobre el clic y, en $R = 0$, sobre la compra, la escala absoluta de τ no es comparable entre estratos. Por eso el estimando principal es relativo y dentro de la búsqueda: el odds ratio condicional $\psi = e^{\gamma}$ del logit condicional (McFadden 1974)

$$P\big(Y_{is} = 1 \mid \textstyle\sum_j Y_{js} = 1,\ T_s,\ Z_s\big) = \frac{\exp(\gamma\, T_{is} + Z_{is}^{\top}\delta)}{\sum_{j \in s} \exp(\gamma\, T_{js} + Z_{js}^{\top}\delta)}, \tag{2}$$

que condiciona en el número de reservas de la lista: el término de cada búsqueda (intención, destino, fecha) se cancela, y ψ no depende de cuántas listas con compra seleccionó el proveedor (lógica de caso-control, Prentice y Pyke (1979)).

2.3. Supuestos de identificación

1. **SUTVA.** El resultado de un hotel depende solo de su propia exposición. Es una aproximación: con slots de suma cero, subir a un hotel desplaza a otro (interferencia). Interpretamos τ como "efecto dada la competencia de la lista".
2. **Positividad.** $0 < e(z) = P(T = 1 \mid Z = z) < 1$. Se verifica con la propensión estimada (sección 7.3).
3. **Intercambiabilidad condicional (Backdoor).** $Y(t) \perp T \mid Z$. Con $R = 1$ el orden aleatorio la garantiza por diseño, salvo la asociación residual que documentamos. Con $R = 0$ vale solo si Z bloquea todo camino trasero; el DAG predice que U lo impide, y el benchmark lo contrasta.

2.4. Diseño muestral y plan experimental

- **Muestra.** De 9.917.530 filas y 399.344 búsquedas de `train.csv` (Liu et al. 2013), tomamos 30.000 búsquedas **completas** (739.323 filas), estratificadas por `random_bool` y con semilla fija. Se muestrea por búsqueda para no romper las listas.
- **Partición.** 80/20 agrupada por búsqueda (ninguna lista queda partida) y estratificada por `random_bool`; validación cruzada `StratifiedGroupKFold(5)` dentro de Train. La partición depende solo del identificador de búsqueda y de la semilla, así que se fija antes de transformar los datos: cercas de Tukey, cuantiles de winsorización y medianas de imputación se ajustan **solo con Train** (tests marcados `leakage` en el Quality Gate).
- **Nivel de significación y potencia.** Fijamos $\alpha = 0{,}05$ a priori. Con n en miles, casi todo contraste rechaza; por eso reportamos, junto con cada p-valor, el tamaño de efecto y el **efecto mínimo detectable** con potencia $1 - \beta = 0{,}80$: $\text{MDE} = (z_{1-\alpha/2} + z_{1-\beta})\sqrt{1/n_1 + 1/n_2}$ en unidades de desvío.

3. Datos y auditoría de calidad

Tabla 2: Centinelas (0 = sin dato): distorsión antes y después de pasarlos a faltante.

	% filas	media antes	media después	DE antes	DE después	sesgo de la media (%)
prop_review_score	5,171	3,758	3,963	1,082	0,649	-5,179
prop_starrating	3,373	3,186	3,297	1,050	0,880	-3,373
prop_log_historical_price	14,826	4,305	5,054	1,860	0,525	-14,826

Hallazgos de la auditoría forense (detalle en el notebook, 1.2):

- **Centinelas.** Tres variables codifican "sin dato" con 0 (Tabla 2). El caso más grave es `prop_log_historical_price`: el 0 aparece en el 14,8 % de las filas y su desvío cae de 1,860 a 0,525 al tratarlo.
- **Desplazamiento de escala del precio.** El 3,2 % de las reservas de dos o más noches tiene el precio por estadía, no por noche, y la proporción varía entre sitios. Se neutraliza con el precio relativo a la mediana de la búsqueda.
- **Valores extremos.** 1.449 filas (0,196 %) caen fuera de las cercas extremas de Tukey sobre el log del precio, [8,43; 1.947,77] USD (ajustadas en Train), o a más de 10 veces la mediana de su búsqueda.
- **Slots estructurales.** Las posiciones 5, 11, 17, 23 casi nunca están en el dataset: las ocupan listados que Kaggle no publica.
- **Selección del dataset.** Ninguna búsqueda tiene cero clics, y el 94 % de las búsquedas con orden del algoritmo tiene una reserva, contra el 13 % con orden aleatorio. Es el colisionador natural de la sección 7.7.

Tabla 3: Faltantes: porcentaje, mayor dependencia con variables observadas y tratamiento (medianas ajustadas solo con Train).

	% faltante	máx. |efecto|	mecanismo	tratamiento
prop_location_score2	22,04	0,51	MAR	mediana por país + indicador

Tabla 3: Faltantes: porcentaje, mayor dependencia con variables observadas y tratamiento (medianas ajustadas solo con Train).

	% faltante	máx. |efecto|	mecanismo	tratamiento
orig_destination_distance	32,80	0,30	MAR	mediana por par de países + indicador
prop_review_score	5,34	0,24	MNAR estructural	mediana por estrellas + indicador
visitor_hist_starrating	95,52	0,09	MNAR estructural	no se imputa (indicador)
srch_query_affinity_score	94,30	0,18	MNAR estructural	no se imputa (indicador)

Mecanismo de los faltantes. Sea $R_j = 1\{X_j \text{ falta}\}$. Bajo MCAR, R_j es independiente de toda variable observada. Lo contrastamos variable por variable (χ^2 o Mann-Whitney con Bonferroni, y tamaño de efecto) y en forma conjunta con el test de Little (Little 1988): d^2 = 75.952,2 con 73 gl, p < 10^{-300} (16 patrones de faltantes). **Se rechaza MCAR.** Las variables con dependencia apreciable de lo observado se tratan como MAR (imputación por grupo con indicador); las MNAR estructurales no se imputan, porque imputar un historial que no existe inventaría información (Tabla 3).

4. Fase 1: inferencia clásica y ANOVA

Contraste 1: sesgo de posición en el clic (pareado, orden aleatorio). Para cada búsqueda s con $R = 1$, $D_s = \overline{\text{CTR}}_{\text{top-5},s} - \overline{\text{CTR}}_{\text{resto},s}$. $H_0 : \mu_D = 0$ frente a $H_1 : \mu_D \neq 0$; t de Student pareado, con Wilcoxon de resguardo.

- $t(14.982)$ = 37,47, p = $7{,}74 \cdot 10^{-294}$; $\bar{D}$ = 0,0561 (IC 95 % [0,0531; 0,0590]); d_z = 0,306; Wilcoxon p = $7{,}70 \cdot 10^{-131}$.
- **Se rechaza** H_0**.** La tasa de clic del top-5 es 2,16 veces la del resto. El MDE con n = 14.983 búsquedas es d_z = 0,023, muy por debajo del efecto observado. Es el arco $T \to M$ del DAG.

Contraste 2: gasto por reserva, usuarios recurrentes vs. nuevos (Welch). Sobre el log del gasto, $H_0 : \mu_{\text{rec}} = \mu_{\text{nue}}$. Usamos Welch aunque Levene no rechaza, porque los grupos están muy desbalanceados (1.025 vs. 15.039).

- $t(1.158{,}6)$ = 1,303, p = 0,193; cociente de medias geométricas 1,039 (IC 95 % [0,981; 1,101]); g de Hedges = 0,043; Mann-Whitney p = 0,093.
- **No se rechaza** H_0**.** No es evidencia de igualdad: la muestra no distingue diferencias menores que g = 0,090 (MDE con potencia 0,80).

ANOVA de un factor: log del precio por noche según estrellas (4 grupos). $H_0 : \mu_{1\text{-}2} = \mu_3 = \mu_4 = \mu_5$.

- **Normalidad.** Con n grande, Shapiro-Wilk rechaza ante desvíos irrelevantes: su potencia tiende a 1 para cualquier alternativa fija, mientras que el estadístico F depende de las medias muestrales, cuya distribución se acerca a la normal por el TCL (la asimetría de $\bar{x}$ decae como $\gamma_1/\sqrt{n}$). Evaluamos asimetría, curtosis y Q-Q por grupo: asimetrías dentro de $[-1, 1]$ y colas algo pesadas (notebook, 1.8).

- **Homocedasticidad.** Levene (Brown-Forsythe) $F(3; 16.060) = 56{,}57$, p = $2{,}27 \cdot 10^{-36}$: **se rechaza** la igualdad de varianzas.
- **Inferencia principal: ANOVA de Welch** (Welch 1951): $F(3; 4.432{,}0) = 1.806{,}9$, p < 10^{-300}, η^2 = 0,254. Kruskal-Wallis coincide: $H(3) = 4.150{,}0$, $\varepsilon^2 = 0{,}258$.
- **Post-hoc.** Games-Howell (Games y Howell 1976) como principal (varianzas desiguales); Tukey HSD y Bonferroni como comparación: las tres coinciden en todos los pares. Un hotel de 5 estrellas cuesta 2,65 veces uno de 1-2 estrellas (medias geométricas).

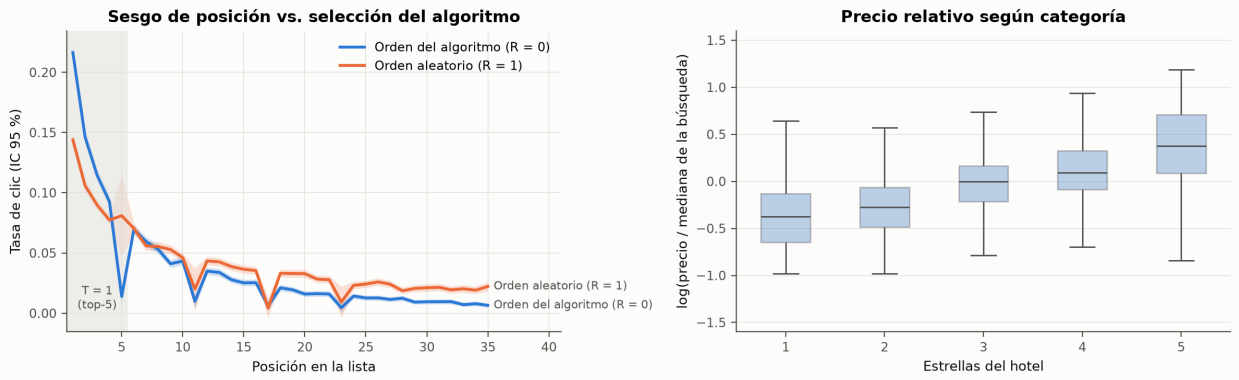


Figura 1: Sesgo de posición: tasa de clic por posición con orden del algoritmo y aleatorio (IC 95 %), y precio relativo por categoría.

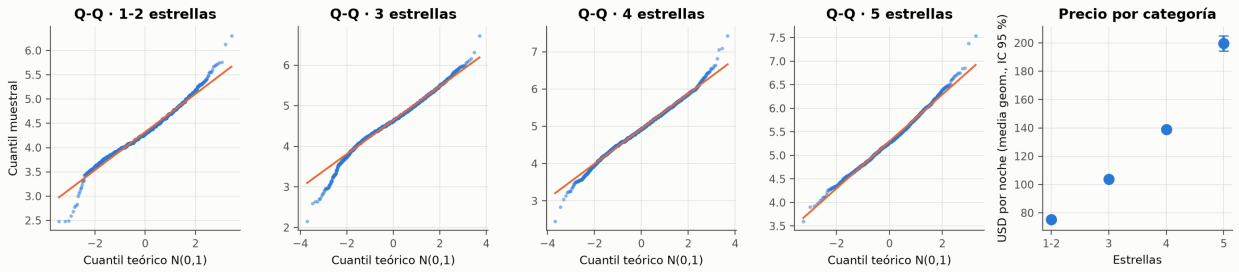


Figura 2: ANOVA del log precio por estrellas: distribuciones, Q-Q por grupo y medias con IC 95 %.

5. Fase 2: Gauss-Markov y PCA

El modelo de referencia es log(precio por noche) $= \beta_0 + \sum_k \beta_k X_k + \varepsilon$ sobre 12.858 reservas de Train. Estimado por MCO: R^2 = 0,604, R^2 ajustado = 0,603, $F(17; 12.840) = 1.150{,}9$. La especificación ingenua sobre el precio crudo fracasa: R^2 = 0,0028, con una sola observación de distancia de Cook 1,215 (> 0,5) que domina el ajuste (BITÁCORA, Entrada 3).

Tabla 4: Auditoría de Gauss-Markov del MCO de referencia.

Supuesto	Prueba y resultado	Remediación
Homocedasticidad	Breusch-Pagan (Breusch y Pagan 1979) LM = 515,9 (gl 17), p = $8{,}34 \cdot 10^{-99}$; White p = $1{,}09 \cdot 10^{-117}$	Errores HC3 (MacKinnon y White 1985)
Normalidad	Jarque-Bera = 4.235,6, p < 10^{-300}; curtosis 5,81	TCL para t y F; Huber (Huber 1964) de resguardo

Supuesto	Prueba y resultado	Remediación
Independencia	Durbin-Watson = 1,952 (orden temporal)	Ninguna
Atípicos y apalancamiento	121 con $\|r_i\| > 3$; 841 con $h_{ii} > 2(p+1)/n$	Se conservan: cola pesada real
Influyentes	748 con $D_i > 4/n$; 0 con $D_i > 0{,}5$ (máx. 0,0117) (Cook 1977)	Huber coincide con MCO sin influyentes
Multicolinealidad	VIF máx. 1,74 (modelo) y 2,98 (Fase 3)	Ninguna: ningún VIF supera 5

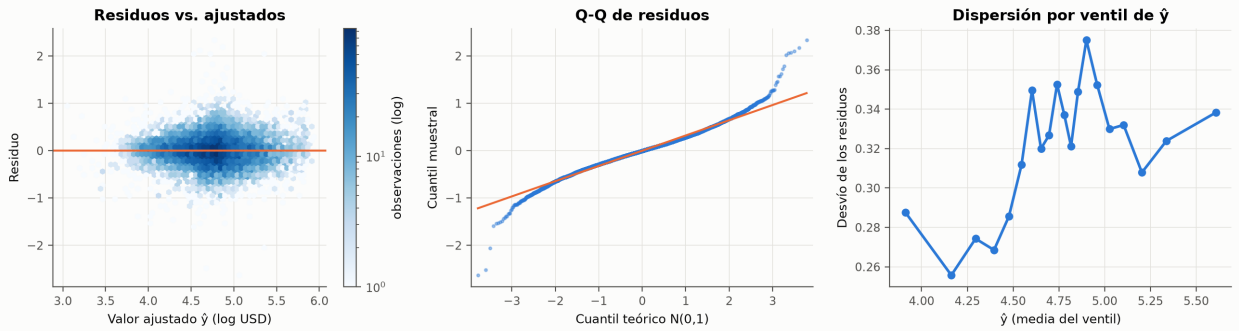


Figura 3: Diagnóstico de residuos: residuos vs. ajustados, Q-Q y dispersión por cuantil de ŷ.

PCA sin fuga. Estandarización y descomposición espectral se ajustan **solo con Train**; Test se proyecta con la misma rotación. Kaiser ($\lambda_k > 1$) retiene 4 componentes (69,7 % de la varianza); el codo sugiere 2. PC1 ordena los hoteles de económicos a premium (estrellas, precio histórico, reseñas); PC2 captura la buena ubicación a buen precio. En Test, los hoteles reservados se separan por PC2 y no por PC1: el usuario premia la oferta conveniente más que la categoría (Figura 4).

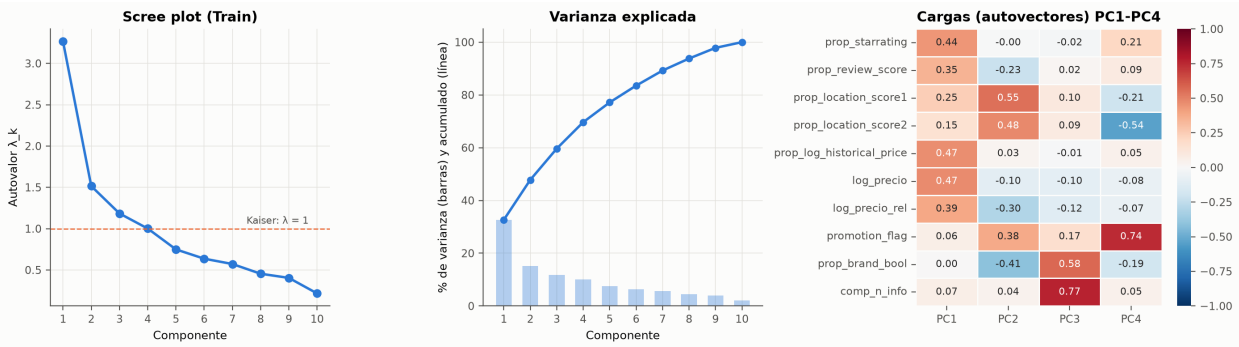


Figura 4: Scree plot, varianza explicada acumulada y cargas de PC1 y PC2 (ajustado en Train).

6. Fase 3: torneo predictivo

Cuatro familias, misma partición y CV agrupada; PR-AUC como métrica de selección (prevalencia ≈ 2 %) y umbral elegido en Train por máximo F1.

Tabla 5: Desempeño en Test (umbral elegido en Train por máximo F1).

	ROC_AUC	PR_AUC	Brier	Precision	Recall	F1	F1_macro	F1_ponderado
Logística	0,844	0,135	0,020	0,168	0,304	0,216	0,596	0,959
Naive Bayes	0,796	0,074	0,070	0,069	0,003	0,006	0,497	0,967
CART podado	0,823	0,120	0,020	0,151	0,379	0,216	0,593	0,953
Random Forest	0,854	0,153	0,020	0,163	0,390	0,230	0,601	0,955

- **Logística regularizada.** La grilla elige penalización L1 con $C = 0.1$. Los odds ratios (Tabla 6) salen de una logística sin penalizar con errores por clúster de búsqueda; son **asociativos**, no causales.
- **Naive Bayes gaussiano.** Ordena aceptablemente (ROC-AUC 0,796) pero sus probabilidades son inutilizables (Brier 0,070): la independencia condicional $P(X \mid Y) = \prod_j P(X_j \mid Y)$ se viola (precio y precio histórico correlacionan dentro de cada clase), cuenta varias veces la misma evidencia y empuja las probabilidades a 0 o a 1 (Niculescu-Mizil y Caruana 2005).
- **CART.** Sin restricciones, el árbol (18.863 hojas) memoriza Train (PR-AUC 1,000) y fracasa en Test (0,031). Con poda por complejidad $R_\alpha(T) = R(T) + \alpha|T|$ y α_{opt} por CV queda en 14 hojas y profundidad 6; la regla raíz separa por posición.
- **Random Forest** (Breiman 2001). Búsqueda aleatoria sobre los cinco hiperparámetros (`n_estimators` = 200, `max_depth` = 24, `min_samples_leaf` = 50, `max_features` = log2). El error OOB (PR-AUC 0,151) converge con la CV. Un holdout cronológico (último 20 % del período) da PR-AUC 0,144, cerca del aleatorio.

Tabla 6: Odds ratios de la logística sin penalizar (Train, errores por clúster de búsqueda): las 12 variables con menor p-valor.

	OR	IC95_inf	IC95_sup	p
position	0,899	0,896	0,902	$p < 10^{-300}$
random_bool	0,146	0,139	0,154	$p < 10^{-300}$
prop_location_score2	4,381	4,016	4,780	$p = 8{,}43 \cdot 10^{-243}$
log_precio_rel	0,399	0,377	0,423	$p = 7{,}59 \cdot 10^{-214}$
prop_location_score2_faltante	0,649	0,616	0,684	$p = 3{,}36 \cdot 10^{-58}$
prop_review_score	1,296	1,252	1,341	$p = 9{,}82 \cdot 10^{-50}$
prop_location_score1	0,915	0,904	0,926	$p = 6{,}48 \cdot 10^{-47}$
log_precio	0,755	0,726	0,785	$p = 5{,}14 \cdot 10^{-45}$
prop_starrating	1,177	1,148	1,207	$p = 8{,}21 \cdot 10^{-37}$
srch_length_of_stay	0,954	0,947	0,961	$p = 2{,}66 \cdot 10^{-35}$
srch_booking_window	0,999	0,998	0,999	$p = 3{,}08 \cdot 10^{-21}$
con_afinidad	1,176	1,136	1,218	$p = 5{,}87 \cdot 10^{-20}$

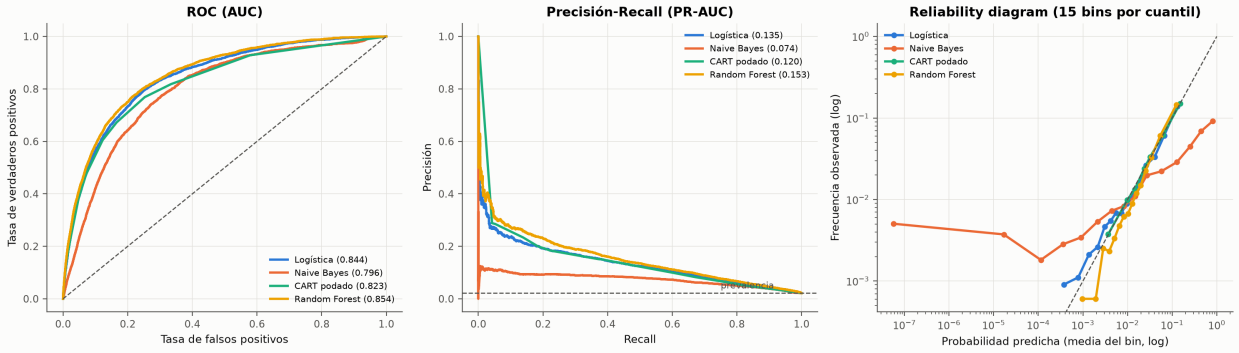


Figura 5: Curvas ROC y PR y diagramas de confiabilidad en Test.

MDI vs. permutación. El MDI suma reducciones de impureza en cada split. Una variable continua con muchos valores distintos ofrece muchos más cortes candidatos que una dummy, así que gana splits por azar, sobre todo en nodos profundos con pocas observaciones, y acumula importancia aunque no generalice (Strobl et al. 2007). Además se calcula en Train. La importancia por permutación mide cuánto cae la PR-AUC **en Test** al romper la relación de una variable con Y, así que no premia los cortes espurios. En nuestros datos, el MDI correlaciona con la cardinalidad de las variables (ρ = 0,70); por permutación, la variable más importante es position (ΔPR-AUC = 0,099).

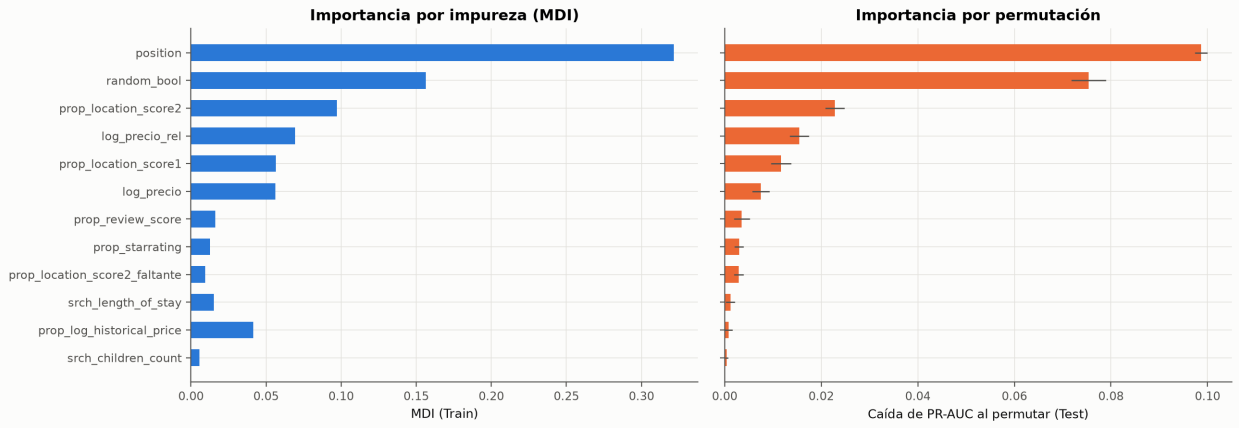


Figura 6: Importancia por MDI (Train) vs. permutación (Test).

7. Fase 4: inferencia causal

7.1. DAG e identificación

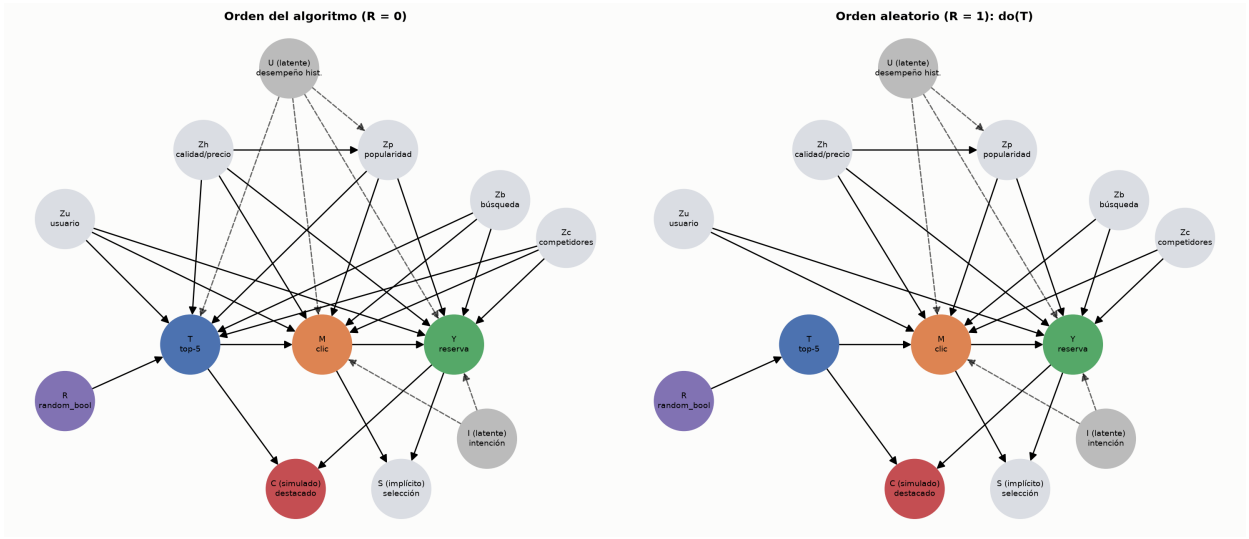


Figura 7: DAG con el orden del algoritmo (izquierda) y con orden aleatorio (derecha, cirugía de grafo de do(T)).

Demostración Backdoor. Con $R = 0$, todo camino trasero entre T e Y entra a T por Z o por U. Z bloquea los caminos $T \leftarrow Z \rightarrow Y$ y no abre ninguno nuevo: no contiene colisionadores ni descendientes de T (el clic M y la selección S quedan fuera del ajuste). Pero $T \leftarrow U \rightarrow Y$ queda abierto: **Backdoor no se cumple si U existe**. Con $R = 1$, la cirugía de grafo elimina los arcos que entran a T salvo $R \rightarrow T$ y el conjunto vacío alcanza. Lo verifican 12 pruebas de d-separación (`tests/test_dag.py`).

7.2. DoWhy en cuatro pasos

Model (`CausalModel` con el DAG y el clic como mediador) → *Identify* (`identify_effect`, Backdoor con Z; DoWhy excluye el mediador) → *Estimate* (regresión lineal backdoor e IPW) → *Refute* (sección 7.8) (Sharma y Kiciman 2020).

Tabla 7: Efecto del top-5 sobre P(reserva) en escala absoluta (DoWhy) y sesgo de confusión del ingenuo.

	ingenuo Y ~ T	regresión backdoor	IPW	tasa base	sesgo (ingenuo − regresión)	sesgo (%)
R = 0 (algoritmo)	0,1055	0,0961	0,0890	0,0206	0,0094	9,7451
R = 1 (aleatorio)	0,0105	0,0092	0,0079	0,0037	0,0013	14,5225

Con el orden del algoritmo, el coeficiente ingenuo de $Y \sim T$ supera al ajustado: el sesgo de confusión es **positivo** (el algoritmo pone arriba a hoteles que se reservarían igual) y equivale al 9,7 % del efecto ajustado. Las magnitudes absolutas **no** son comparables entre estratos, porque la selección del dataset infla la tasa base del orden del algoritmo.

7.3. Positividad y overlap

Tabla 8: Diagnóstico de positividad: propensión estimada ê(z) por grupo, fracción fuera de [0,01; 0,99], peso máximo y tamaño efectivo de Kish.

	R = 0 (algoritmo)	R = 1 (aleatorio)
ê p1 (top-5)	0,037	0,061
ê p99 (top-5)	0,697	0,479
ê p1 (resto)	0,026	0,056
ê p99 (resto)	0,471	0,342
% fuera de [0,01; 0,99] (top-5)	0,002	0,000
% fuera (resto)	0,010	0,000
peso IPW máximo	119,955	30,877
ESS top-5	35.929,411	50.156,063
n top-5	60.265,000	59.864,000
ESS resto	308.341,248	301.424,690
n resto	315.932,000	303.262,000

El IPW identifica τ solo si se cumple la positividad (Rosenbaum y Rubin 1983). Las propensiones del top-5 y del resto se superponen en todo su rango y casi ninguna unidad cae fuera de [0,01; 0,99] (Tabla 8, Figura 8). El IPW de Hájek da 0,0882 (IC 95 % bootstrap por búsqueda [0,0854; 0,0909]), y excluyendo las unidades sin overlap (Crump et al. 2009), 0,0882 [0,0854; 0,0910]: el recorte no cambia la estimación. El IC fija la propensión estimada, lo que lo hace conservador (Hirano et al. 2003). **La positividad no es el problema de este trabajo; *U* sí**: ningún reponderado por lo observado corrige la confusión que revela el benchmark.

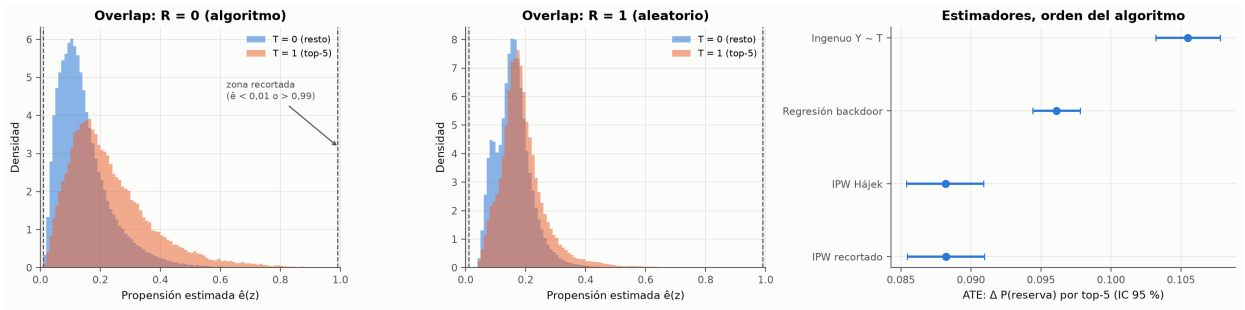


Figura 8: Overlap de la propensión por grupo en cada estrato (zonas recortadas sombreadas) y estimadores del ATE con IC 95 % (orden del algoritmo).

7.4. Escala relativa: observacional vs. cuasi-experimento

Tabla 9: OR intra-búsqueda del top-5 (logit condicional, ecuación 2) y diferencia de probabilidad con efectos fijos (MPL-EF).

	OR intra	IC 95 %	búsquedas informativas	Δp (MPL-EF)
R = 0 (algoritmo) | sin ajuste	5,383	[5,196; 5,576]	14.124	0,0973
R = 0 (algoritmo) | ajustado por Z	3,378	[3,242; 3,519]	14.124	0,0778

Tabla 9: OR intra-búsqueda del top-5 (logit condicional, ecuación 2) y diferencia de probabilidad con efectos fijos (MPL-EF).

	OR intra	IC 95 %	búsquedas informativas	Δp (MPL-EF)
R = 1 (aleatorio) | sin ajuste	3,002	[2,717; 3,316]	1.971	0,0085
R = 1 (aleatorio) | ajustado por Z	2,112	[1,895; 2,355]	1.971	0,0064

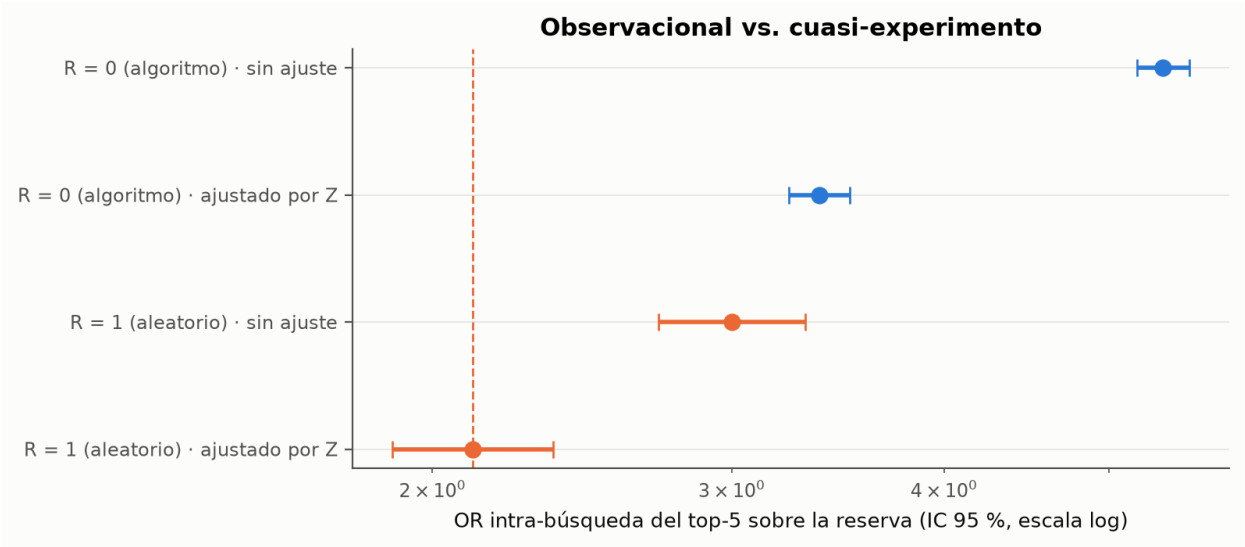


Figura 9: OR intra-búsqueda: observacional (orden del algoritmo) vs. benchmark (orden aleatorio).

La brecha entre el OR ingenuo del algoritmo y el benchmark es de 0,935 en log-odds; el ajuste por Z explica el 49,8 %. El resto es compatible con la confusión no observada U. La sensibilidad con T = top-4 del dataset (los slots fijos no publicados no cuentan) da OR = 2,128 [1,909; 2,372].

7.5. Robustez de la conclusión central

La afirmación "el observacional ajustado sobreestima el efecto causal" se sometió a tres pruebas (notebook, 4.3 bis), sobre las búsquedas con una reserva:

1. **Balance de poblaciones.** La mayor |SMD| entre búsquedas del algoritmo y aleatorias es 0,248; con pesos de odds baja a 0,005 (tamaño efectivo 12.938 de 14.124). Re-ponderado, el cociente de OR es 1,594: la brecha no es una diferencia de población.
2. **No colapsabilidad del OR.** En diferencia de riesgos (efecto marginal promedio del logit de elección), el top-5 suma 5,34 p.p. con el algoritmo contra 3,24 p.p. con orden aleatorio.
3. **Incertidumbre.** Wald de igualdad de log-OR: z = 7,92, p = $2{,}34 \cdot 10^{-15}$. Bootstrap percentil por búsqueda con 1000 réplicas (Tabla 10): ninguna réplica muestra un cociente ≤ 1.

Tabla 10: Bootstrap por búsqueda (IC 95 % percentil): OR, efectos marginales (AME) y diferencia de riesgos con efectos fijos (DR-EF).

	IC95_inf	mediana	IC95_sup
OR_R0	3,2508	3,3824	3,5196
OR_R1	1,8952	2,1110	2,3636
cociente_OR	1,4213	1,6018	1,8025
cociente_OR_ponderado	1,4181	1,5976	1,7950
AME_R0	0,0514	0,0535	0,0555
AME_R1	0,0272	0,0324	0,0378
cociente_AME	1,4062	1,6525	1,9776
diferencia_AME	0,0153	0,0210	0,0269
cociente_AME_ponderado	1,4477	1,7011	2,0336
DR_EF_R0	0,0801	0,0826	0,0852
DR_EF_R1	0,0410	0,0477	0,0548
diferencia_DR_EF	0,0277	0,0350	0,0423
cociente_DR_EF	1,5056	1,7351	2,0137

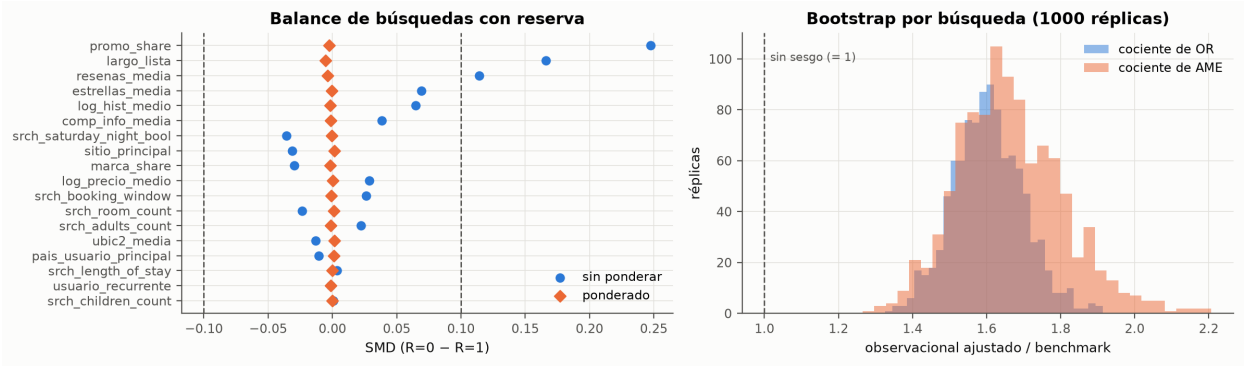


Figura 10: Robustez: balance antes y después de re-ponderar y distribución bootstrap de los cocientes.

7.6. Mediador: el clic como paso del embudo

Reservar implica hacer clic, así que la identidad $\mathrm{RR}_Y = \mathrm{RR}_{\text{clic}} \times \mathrm{RR}_{\text{conversión|clic}}$ descompone el efecto. Con orden aleatorio, $\mathrm{RR}_{\text{clic}}$ = 2,96 [2,874; 3,058] y $\mathrm{RR}_{\text{conversión|clic}}$ = 1,30 (descriptivo: condiciona en un colisionador). El 80 % del efecto en escala log opera a través de más clics.

Trampa del mediador, demostrada. Con el orden del algoritmo, el efecto total ajustado por Z es Δp = 0,0778. Si además se ajusta por el clic, cae a 0,0042: el modelo concluye que la posición casi no importa, porque bloquea el canal por el que opera. Ajustar por M estima solo el efecto directo y subestima el impacto total de la política.

7.7. Trampas de selección y del colisionador

Tabla 11: Trampas demostradas: ajustar por el mediador, condicionar en la selección S y en el colisionador C.

	Δp	IC 95 %	p
efecto total (ajuste Z)	0,0778	[0,0753; 0,0803]	$p < 10^{-300}$
ajustando además por el clic (M)	0,0042	[0,0031; 0,0052]	$p = 9{,}86 \cdot 10^{-16}$
solo hoteles clickeados (condicionar en M)	0,1150	[0,0452; 0,1847]	p = 0,001
simulación: todas las listas	0,0338	[0,0302; 0,0373]	—
simulación: solo listas con ≥ 1 evento (S = 1)	0,0684	[0,0615; 0,0752]	—
datos R = 1: todas las filas (R = 1)	0,0105	[0,0096; 0,0115]	—
datos R = 1: condicionando en C = 1	0,0136	[0,0096; 0,0177]	—

- **Colisionador natural (selección S).** En una simulación con verdad conocida, el OR intra-búsqueda se recupera igual con y sin el filtro "≥ 1 evento por lista" (2,320 en ambos casos), mientras que la diferencia absoluta se duplica: por eso el estimando principal es relativo.
- **Colisionador simulado** C ($T \rightarrow C \leftarrow Y$). Condicionar en C induce una asociación espuria (sesgo de Berkson): sobre los datos con orden aleatorio, la Δp ingenua pasa de 0,0105 a 0,0136, y en la simulación con verdad conocida el signo se invierte. La única regla segura es no condicionar.

7.8. Refutaciones

Tabla 12: Refutaciones de DoWhy sobre la regresión backdoor (orden del algoritmo).

	ATE original	ATE refutado	p	pasa
Causa común aleatoria	0,0961	0,0961	p = 0,485	sí
Placebo (T permutado)	0,0961	-0,0002	p = 0,416	sí
Subconjunto 80 %	0,0961	0,0962	p = 0,443	sí

- **Causa común aleatoria:** agregar al DAG una variable independiente no mueve el ATE: la estimación no depende de detalles accidentales del conjunto de ajuste.
- **Placebo:** con T permutado, el ATE cae a ≈ 0 y no se rechaza que sea nulo: el efecto no es un artefacto del procedimiento.
- **Subconjunto del 80 %:** el ATE queda dentro del IC original: la estimación es estable.

Las tres miden **estabilidad**, no validez: no detectan la confusión por U. Esa la revela el benchmark con orden aleatorio, que es la refutación más exigente del trabajo. La sensibilidad a un U simulado (notebook, 4.6) muestra que una confusión de fuerza moderada alcanza para anular el efecto absoluto de $R = 0$.

7.9. Double Machine Learning y CATE

DML (Chernozhukov et al. 2018) estima con modelos de ruido no lineales (Random Forest) $\hat{m}(W, X) = \mathbb{E}[Y \mid W, X]$ y $\hat{e}(W, X) = P(T = 1 \mid W, X)$ con *cross-fitting* por búsqueda, y regresa el residuo de Y sobre el residuo de T:

$$Y - \hat{m}(W, X) = \tau(X)\left(T - \hat{e}(W, X)\right) + \varepsilon. \tag{3}$$

La ortogonalidad de Neyman hace que los errores de $\hat{m}$ y $\hat{e}$ afecten a $\hat{\tau}$ solo en segundo orden. `LinearDML` (Battocchi et al. 2019) estima un ATE de 0,0833 [0,0789; 0,0877] con el orden del algoritmo y de 0,0073 [0,0059; 0,0088] con orden aleatorio; relativos a sus tasas base, los cocientes son 4,12 y 2,01: el mismo patrón de sobreestimación que el logit condicional.

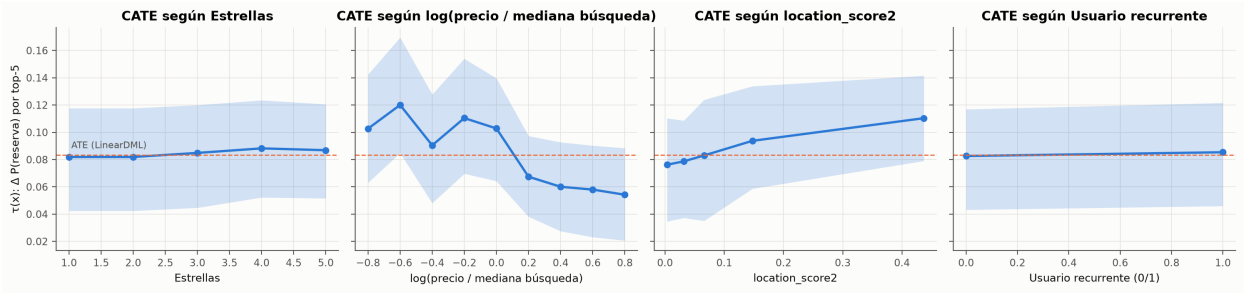


Figura 11: CATE del top-5 (Causal Forest DML, orden del algoritmo) según estrellas, precio relativo, ubicación y usuario recurrente, con bandas de IC 95 %.

Tabla 13: Heterogeneidad con orden aleatorio: LinearDML, $\tau(x) = \theta_0 + x^T\theta$ (R = 1).

	θ (efecto de x sobre τ)	IC95_inf	IC95_sup	p
prop_starrating	0,0006	-0,0012	0,0025	p = 0,489
log_precio_rel	-0,0049	-0,0085	-0,0012	p = 0,009
prop_location_score2	0,0124	0,0012	0,0235	p = 0,030
usuario_recurrente	0,0123	0,0000	0,0246	p = 0,050

El Causal Forest (Wager y Athey 2018; Athey et al. 2019) sobre los datos del algoritmo sugiere que el slot rinde más para hoteles baratos para su búsqueda y bien ubicados (Figura 11). La heterogeneidad **se confirma con orden aleatorio**: los coeficientes del LinearDML de $R = 1$ tienen el mismo signo (Tabla 13), así que no es un artefacto de U. Figura 12 cuantifica el efecto por segmento.

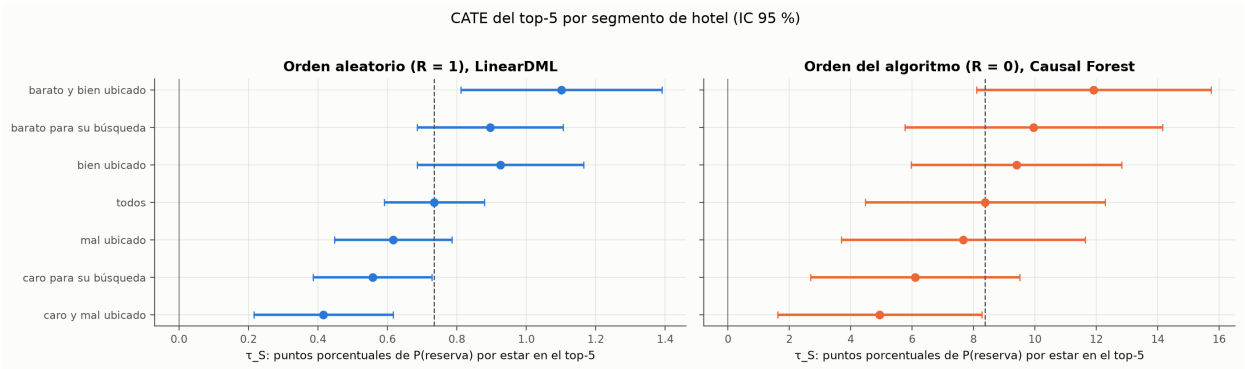


Figura 12: CATE del top-5 por segmento de hotel (terciles de precio relativo y de ubicación), con IC 95 %: orden aleatorio (LinearDML) y orden del algoritmo (Causal Forest).

8. Ensayo crítico: capacidad predictiva vs. palanca causal

¿Si la posición es la variable más importante del Random Forest, puede la dirección usarla como palanca? En la Fase 3, la posición es la variable con mayor importancia por permutación en Test y la regla raíz del árbol podado. La tentación es leer el OR ingenuo (5,38) como lo que gana un hotel al subirlo. La Fase 4 muestra por qué es un error.

1. **Gran parte de esa capacidad predictiva es selección, no causa.** El algoritmo pone arriba a los hoteles que se iban a reservar igual, por atributos que observamos y por un desempeño histórico que no. Para *predecir*, la posición es útil justamente porque resume todo eso. Para *intervenir*, solo cuenta el efecto que queda cuando la selección se rompe: el orden aleatorio da un OR de 2,11.
2. **La palanca existe, pero es más chica y heterogénea.** Mover un hotel al top-5 sí aumenta sus reservas, y más si es barato para su búsqueda y está bien ubicado.
3. **Cuándo una variable muy predictiva no tiene poder causal, o tiene el signo opuesto:**
   - cuando es un **proxy de confusores** (la posición resume U);
   - cuando es **consecuencia del resultado o un colisionador** (C, o el clic si se condiciona en él): condicionar en C puede invertir el signo del efecto;
   - cuando hay **selección de la muestra** sobre el resultado (S): `random_bool` es la segunda variable más predictiva del Random Forest y no es una palanca de negocio, sino la huella de cómo el proveedor seleccionó cada estrato.

   El mismo dato da tres respuestas distintas según qué se condiciona: el modelo que "controla" por el clic concluye que la posición no importa; el que condiciona en C, que perjudica; el que ajusta por Z con el orden del algoritmo, que importa más de lo que importa.

Conclusión del ensayo. La importancia predictiva responde "¿qué variables ayudan a adivinar Y?"; la palanca causal responde "¿qué pasa con Y si muevo X?". Solo el DAG, la identificación y, cuando existe, un experimento, permiten pasar de una a la otra (Pearl 2009; Hernán y Robins 2020).

9. Recomendaciones de negocio cuantificadas

Tabla 14: Valuación del slot por escenarios (por 1.000 exposiciones en el top-5; búsquedas con al menos un clic).

	Δp por exposición	gasto por reserva (USD)	fracción desplazada	reservas / 1.000	USD / 1.000
conservador	0,005	223,270	0,500	2,750	613,992
central	0,006	223,270	0,000	6,441	1.438,160
optimista	0,011	383,486	0,000	10,542	4.042,859

Tabla 15: Efecto causal por segmento con orden aleatorio (LinearDML, R = 1): reservas y USD incrementales por 1.000 exposiciones (gasto mediano, sin desplazamiento).

	reservas / 1.000	IC 95 % inf.	IC 95 % sup.	USD / 1.000	relativo a todos
barato y bien ubicado	11,01	8,11	13,91	2.458,41	1,50

Tabla 15: Efecto causal por segmento con orden aleatorio (LinearDML, R = 1): reservas y USD incrementales por 1.000 exposiciones (gasto mediano, sin desplazamiento).

	reservas / 1.000	IC 95 % inf.	IC 95 % sup.	USD / 1.000	relativo a todos
barato para su búsqueda	8,96	6,86	11,07	2.001,11	1,22
bien ubicado	9,26	6,86	11,65	2.066,72	1,26
todos	7,35	5,91	8,79	1.640,31	1,00
mal ubicado	6,16	4,48	7,85	1.376,16	0,84
caro para su búsqueda	5,57	3,86	7,29	1.243,86	0,76
caro y mal ubicado	4,16	2,16	6,16	928,84	0,57

1. **Valuar el slot con el efecto causal, en un rango.** Cada 1.000 exposiciones en el top-5 valen entre 614 y 4.043 USD de gasto incremental (Tabla 14), según el efecto supuesto, el gasto por reserva y cuánto de la reserva ganada se le quita a otro hotel de la lista. Sirve para ordenar decisiones y negociar contratos de posicionamiento, no para presupuestar: la escala absoluta depende de la selección del dataset.
2. **No pagar el sesgo.** Una valuación con datos del algoritmo, aun ajustada, sobrestima el OR del slot en 60 %. Un contrato de posicionamiento pago calculado así sobreprecia el inventario en esa proporción.
3. **Asignar la exposición por segmento.** El slot rinde 1,50 veces el promedio con hoteles baratos para su búsqueda y bien ubicados, y 0,57 veces con hoteles caros y mal ubicados (Tabla 15). Reasignar 1.000 exposiciones del segundo grupo al primero suma 6,85 reservas incrementales, unos 1.530 USD con el gasto mediano. Los IC 95 % de los dos segmentos extremos no se superponen entre sí ([8,1; 13,9] contra [2,2; 6,2] reservas por 1.000); los de los segmentos intermedios sí se superponen con el promedio, así que la recomendación se apoya en los extremos.
4. **Experimentar en forma continua.** El orden aleatorio fue la única fuente de verdad que permitió detectar el sesgo. Mantener una fracción chica de tráfico con orden aleatorio funciona como auditoría permanente del valor causal del ranking; su costo es la conversión que se pierde en ese tráfico, y su beneficio, no sobrepagar ni sobrevender el slot.
5. **Límites.** Todo vale para búsquedas con al menos un clic. Con slots de suma cero, el efecto neto para la plataforma es menor que la suma de efectos individuales (el escenario conservador supone que la mitad de las reservas ganadas son desplazadas).

10. Limitaciones y amenazas a la validez

- **Validez interna.** (i) Aleatorización imperfecta: con $R = 1$ queda una asociación residual posición-atributos; se declara y se ajusta, sin forzar una explicación. (ii) Confusión por U en $R = 0$: cuantificada por la brecha con el benchmark, no eliminada. (iii) SUTVA: el estimando es "efecto dada la competencia de la lista".
- **Validez externa.** La población es "búsquedas con al menos un clic" de 2012-2013; el efecto sobre el tráfico total no está identificado. La escala absoluta del estrato del algoritmo está inflada por el sobremuestreo de compras.
- **Validez de constructo.** T se define sobre la posición en pantalla; la sensibilidad con el top-4 del dataset da el mismo OR.
- **Validez de la conclusión estadística.** Con n grande todo es significativo: reportamos tamaños de efecto, MDE e IC bootstrap por búsqueda, no solo p-valores. Los IC de los CATE por segmento

son anchos para los segmentos intermedios.
- **Confidencialidad.** Los resultados publicados podrían considerarse "model parameters" según las reglas de Kaggle: es un riesgo asumido y documentado (BITÁCORA, Entrada 1).

11. Conclusiones

1. **El slot del top-5 tiene efecto causal positivo sobre la reserva**: con orden aleatorio, OR intra-búsqueda 2,11 [1,90; 2,35]. La exposición opera sobre todo a través de más clics.
2. **El dato observacional del algoritmo lo sobreestima aun después de ajustar** por todo lo observable (cociente de OR 1,60, IC bootstrap que no incluye 1). El ajuste por Z explica la mitad del sesgo; la otra mitad es confusión no observada. Backdoor con lo observable no alcanza en este sistema.
3. **El efecto es heterogéneo y la heterogeneidad es causal**: el slot rinde más con hoteles baratos para su búsqueda y bien ubicados, y el orden aleatorio lo confirma.
4. **La variable más predictiva no es una palanca de la misma magnitud.** Un modelo predictivo excelente puede recomendar políticas equivocadas si se lee como causal.
5. **Recomendación:** valuar y asignar el slot con el efecto causal por segmento y sostener una fracción de tráfico con orden aleatorio como auditoría.

Referencias

Anthropic. 2026. *Claude Code*. Asistente de programación agéntico; modelo Claude Opus 5.5. https://www.anthropic.com/claude-code.

Athey, Susan, Julie Tibshirani, y Stefan Wager. 2019. «Generalized Random Forests». *The Annals of Statistics* 47 (2): 1148-78. https://doi.org/10.1214/18-AOS1709.

Battocchi, Keith, Eleanor Dillon, Maggie Hei, et al. 2019. «EconML: A Python Package for ML-Based Heterogeneous Treatment Effects Estimation». https://github.com/py-why/EconML.

Breiman, Leo. 2001. «Random Forests». *Machine Learning* 45 (1): 5-32. https://doi.org/10.1023/A:1010933404324.

Breusch, T. S., y A. R. Pagan. 1979. «A Simple Test for Heteroscedasticity and Random Coefficient Variation». *Econometrica* 47 (5): 1287-94. https://doi.org/10.2307/1911963.

Chernozhukov, Victor, Denis Chetverikov, Mert Demirer, et al. 2018. «Double/Debiased Machine Learning for Treatment and Structural Parameters». *The Econometrics Journal* 21 (1): C1-68. https://doi.org/10.1111/ectj.12097.

Cook, R. Dennis. 1977. «Detection of Influential Observation in Linear Regression». *Technometrics* 19 (1): 15-18. https://doi.org/10.1080/00401706.1977.10489493.

Crump, Richard K., V. Joseph Hotz, Guido W. Imbens, y Oscar A. Mitnik. 2009. «Dealing with Limited Overlap in Estimation of Average Treatment Effects». *Biometrika* 96 (1): 187-99. https://doi.org/10.1093/biomet/asn055.

Expedia, y Kaggle. 2013. *Personalize Expedia Hotel Searches – ICDM 2013*. Competencia de Kaggle. https://www.kaggle.com/c/expedia-personalized-sort.

Games, Paul A., y John F. Howell. 1976. «Pairwise Multiple Comparison Procedures with Unequal n's and/or Variances: A Monte Carlo Study». *Journal of Educational Statistics* 1 (2): 113-25. https://doi.org/10.3102/10769986001002113.

Hernán, Miguel A., y James M. Robins. 2020. *Causal Inference: What If*. Chapman & Hall/CRC.

Hirano, Keisuke, Guido W. Imbens, y Geert Ridder. 2003. «Efficient Estimation of Average Treatment Effects Using the Estimated Propensity Score». *Econometrica* 71 (4): 1161-89. https://doi.org/10.1111/1468-0262.00442.

Huber, Peter J. 1964. «Robust Estimation of a Location Parameter». *The Annals of Mathematical Statistics* 35 (1): 73-101. https://doi.org/10.1214/aoms/1177703732.

Little, Roderick J. A. 1988. «A Test of Missing Completely at Random for Multivariate Data with Missing Values». *Journal of the American Statistical Association* 83 (404): 1198-202. https://doi.org/10.1080/01621459.1988.10478722.

Liu, Xudong, Bing Xu, Yuyu Zhang, et al. 2013. *Combination of Diverse Ranking Models for Personalized Expedia Hotel Searches*. https://arxiv.org/abs/1311.7679.

MacKinnon, James G., y Halbert White. 1985. «Some Heteroskedasticity-Consistent Covariance Matrix Estimators with Improved Finite Sample Properties». *Journal of Econometrics* 29 (3): 305-25. https://doi.org/10.1016/0304-4076(85)90158-7.

McFadden, Daniel. 1974. «Conditional Logit Analysis of Qualitative Choice Behavior». En *Frontiers in Econometrics*, editado por Paul Zarembka. Academic Press.

Niculescu-Mizil, Alexandru, y Rich Caruana. 2005. «Predicting Good Probabilities with Supervised Learning». *Proceedings of the 22nd International Conference on Machine Learning*, 625-32. https://doi.org/10.1145/1102351.1102430.

Pearl, Judea. 2009. *Causality: Models, Reasoning, and Inference*. 2.ª ed. Cambridge University Press. https://doi.org/10.1017/CBO9780511803161.

Prentice, R. L., y R. Pyke. 1979. «Logistic Disease Incidence Models and Case-Control Studies». *Biometrika* 66 (3): 403-11. https://doi.org/10.1093/biomet/66.3.403.

Rosenbaum, Paul R., y Donald B. Rubin. 1983. «The Central Role of the Propensity Score in Observational Studies for Causal Effects». *Biometrika* 70 (1): 41-55. https://doi.org/10.1093/biomet/70.1.41.

Sharma, Amit, y Emre Kiciman. 2020. *DoWhy: An End-to-End Library for Causal Inference*. https://arxiv.org/abs/2011.04216.

Strobl, Carolin, Anne-Laure Boulesteix, Achim Zeileis, y Torsten Hothorn. 2007. «Bias in Random Forest Variable Importance Measures: Illustrations, Sources and a Solution». *BMC Bioinformatics* 8: 25. https://doi.org/10.1186/1471-2105-8-25.

Wager, Stefan, y Susan Athey. 2018. «Estimation and Inference of Heterogeneous Treatment Effects using Random Forests». *Journal of the American Statistical Association* 113 (523): 1228-42. https://doi.org/10.1080/01621459.2017.1319839.

Welch, B. L. 1951. «On the Comparison of Several Mean Values: An Alternative Approach». *Biometrika* 38 (3/4): 330-36. https://doi.org/10.1093/biomet/38.3-4.330.

12. Anexo A: uso de inteligencia artificial generativa

Herramienta. Claude Code (Anthropic 2026), asistente de programación agéntico de Anthropic (modelo Claude Opus 5.5), usado desde VS Code entre el 11 y el 13/11/2026.

Cómo trabajamos con el agente. El asistente trabajó dentro del repositorio con permisos para leer y escribir archivos y ejecutar comandos (Python, `pytest`, `ruff`, Quarto, `git` y `gh`). Cada fase empezó con un prompt de diseño de Verónica (Project Lead) que fijaba el objetivo, los criterios de aceptación y lo que el asistente no podía decidir solo (datos, publicación, merges). El asistente propuso el diseño, escribió el código, los tests, el notebook y los borradores; ejecutó todo localmente y cerró cada fase con una lista de "qué revisar". Verónica revisó cada incremento, aprobó o pidió cambios, y esas revisiones quedaron registradas como decisiones con autor. En la sesión de cierre (13/11) el asistente trabajó en modo autónomo sobre una auditoría previa contra el enunciado y la rúbrica (`docs/estado_cierre.md`).

Prompts de diseño principales (detalle en `docs/ia_log.md`):

1. Brief del TPI: leer enunciado y rúbrica, armar el repositorio, proponer datasets y roles, sin descargar datos antes de la aprobación.
2. Contrato de datos, test de `random_bool` y DAG borrador, todavía sin datos.
3. Descarga verificada, confidencialidad con mitigaciones testeadas y plan B listo para activar.
4. Un prompt por fase: diseño, ejecución, cierre con "qué revisar" y guía de defensa por rol.
5. Revisiones de Verónica que cambiaron el análisis: contrastar $Z_c \to T$ antes de llamarla confusor, transcribir las definiciones de Kaggle, y la revisión de robustez de la conclusión central (balance, diferencia de riesgos, bootstrap con 1000 réplicas, embudo del clic).
6. Prompt de cierre (13/11), transcripto en el recuadro de abajo como ejemplo.

Errores y alucinaciones detectados. El registro documenta 32 errores en las Fases 0-5 y los de la sesión de cierre, cada uno con el test o la guarda que lo refuta o previene. Ejemplos:

Error del asistente	Cómo se detectó	Test o guarda determinística
Afirmó de memoria que `random_bool` define un experimento puro	Verificación en los datos: la asociación posición-atributos persiste con $R = 1$	`xfail` estricto en el test del contrato de datos
Traía de memoria el signo de `compN_rate`	Verónica transcribió la definición de Kaggle	Test del contrato sobre el dominio de `compN_inv`
RANSAC con el umbral por defecto (MAD de Y, no de los residuos)	Coeficientes inestables vs. MCO	Umbral corregido; Entrada 3
Logit condicional sin converger, con un número plausible	Revisión de la salida del optimizador	Guarda que lanza error y test contra el Newton de statsmodels
Redactó un resultado antes de ver el output	Cotejo de cada número contra un output	Todo número del informe se lee de `resultados.json`
Imputación y winsorización ajustadas con toda la muestra (fuga Train/Test), detectado en la auditoría de cierre	Auditoría contra la regla "todo se ajusta en Train"	Test que perturba Test sobre la muestra real, más 4 tests `leakage`
Esperaba que truncar la propensión no sesgara el IPW	El test con verdad conocida falló	Test con verdad conocida: excluir no sesga, truncar sí

Declaración de alcance y auditoría humana. El asistente escribió el código y los borradores de texto; las decisiones de diseño, las aprobaciones y la revisión final son del equipo y constan con autor

en el registro. Ningún número de este informe se tomó del texto del asistente: todos salen de `models/resultados.json`, que genera el notebook ejecutado de punta a punta. Los controles automáticos (tests, guardas de convergencia, chequeo de confidencialidad) atraparon errores de cómputo; las objeciones de fondo (comparabilidad de poblaciones, no colapsabilidad, incertidumbre de un cociente, el clic como paso del embudo) vinieron de la revisión humana. Lucía y Antonella no ejecutaron el trabajo técnico; sus roles se mantienen para la defensa oral, como declara la BITÁCORA.

Prompt de cierre (13/11/2026), texto completo.

```
# Misión: cerrar el TPI de EAyMR 2026 para sacar un 10

Sos el ingeniero responsable de terminar y entregar este proyecto. El trabajo es
 ↪ el
Trabajo Práctico Integrador "Decision Intelligence Capstone" de Estadística
 ↪ Aplicada y
Modelos de Regresión (UCA, 2S2026). Nuestro problema es el **retorno causal de un
algoritmo de recomendación**, con el dataset **Expedia Hotel Search (Kaggle)**.
Herramientas centrales: statsmodels (OLS con ajuste Backdoor y CATE), DoWhy (DAG,
identificación, refutaciones) y EconML (DML).

La entrega y la defensa oral son el **miércoles 18/11/2026 a las 23:59**. Hoy es
 ↪ 13/11.
Al final del día tengo que mandarle los entregables a mis compañeras. El objetivo
 ↪ no es
llegar a "funciona": cada criterio de la rúbrica tiene que quedar en el nivel
**Excelente**. Trabajá de forma autónoma y completa. Solo pará para preguntarme lo
 ↪ que
de verdad no puedas hacer vos (logins, deploy en la nube, decisiones
 ↪ irreversibles).

---

## 0. Fuentes de verdad (leelas COMPLETAS antes de tocar nada)

1. `docs/Enunciado_TP_Integrador_EAyMR_2026_1.pdf`: el enunciado oficial (5
 ↪ fases, entregables, tabla de 100 pts).
2. `docs/rubrica.txt`: la rúbrica complementaria (Analizar / Modelar / Explorar /
 ↪ Informar, de 1 a 4 pts por subcriterio).
3. Todo lo que ya existe: `README.md`, `BITACORA.md`, `docs/*.md`
 ↪ (auditoria_final, dag_borrador, defensa, ia_log, plan_b),
 ↪ `report/informe.qmd`, `report/slides.qmd`, `referencias.bib`, `notebooks/`,
 ↪ `src/`, `app/`, `tests/`, `models/`, `scripts/`, `.github/`,
 ↪ `pyproject.toml`, `requirements.txt`.
4. Estado de git y GitHub: `git status`, `git log --all --oneline --graph`, las 7
 ↪ ramas, el PR abierto y los issues (`gh pr list`, `gh issue list --limit 100`,
 ↪ `gh pr view`). En `main` hay un solo commit, así que probablemente haya
 ↪ trabajo hecho en otras ramas que todavía no se integró. Revisalo bien antes de
 ↪ rehacer cualquier cosa.

**Regla:** no reescribas lo que ya está bien. Primero auditá, después completá.

---
```

```
## 1. Auditoría inicial (sin modificar código)

Generá `docs/estado_cierre.md` con una matriz de trazabilidad. Cada fila es un
 ↪  requisito
del enunciado o un subcriterio de la rúbrica, con estas columnas:
`Requisito | Dónde está (archivo:sección) | Estado (OK / parcial parcial / falta
 ↪  falta) | Qué falta para Excelente | Prioridad`.

Cubrí como mínimo:
- **Fase 1:** EDA (KDE, boxplots, correlaciones Pearson y Spearman), mecanismo de
 ↪  faltantes MCAR/MAR/MNAR justificado (Expedia tiene muchísimos NaN en
 ↪  competidores, historial del usuario y review score; testealo, por ejemplo con
 ↪  un test de Little o con un logit de missingness sobre las otras variables), ≥2
 ↪  contrastes de hipótesis con α fijado a priori, p exacto, IC 95% de la
 ↪  diferencia y tamaño de efecto, ANOVA de 1 factor con ≥3 grupos, normalidad
 ↪  evaluada por skewness, curtosis y Q-Q (con una explicación de por qué Shapiro
 ↪  no sirve con N grande), Levene, Welch o Kruskal-Wallis, post-hoc Tukey vs
 ↪  Bonferroni.
- **Fase 2:** OLS con R², R² ajustado, F y t; Breusch-Pagan o White, Jarque-Bera,
 ↪  Durbin-Watson si corresponde; residuos estudentizados (>3), leverage
 ↪  (>2(p+1)/n), Cook (>4/n y >0.5) y decisión justificada (Huber/RANSAC o HC3);
 ↪  VIF con su estrategia; PCA ajustado SOLO en train (scree, varianza acumulada,
 ↪  codo o Kaiser, interpretación de loadings PC1/PC2).
- **Fase 3:** split 80/20 (estratificado si es clasificación) +
 ↪  StratifiedKFold(5); Logit regularizada con Odds Ratios e IC 95% (o
 ↪  Ridge/Lasso/ElasticNet si es regresión); Naive Bayes con discusión de la
 ↪  independencia condicional y su efecto en la calibración; CART sin
 ↪  restricciones (overfitting train vs test) y después podado con
 ↪  `cost_complexity_pruning_path` + CV, con el árbol visualizado y la regla de la
 ↪  raíz interpretada; Random Forest con Grid o RandomizedSearchCV sobre los 5
 ↪  hiperparámetros pedidos y OOB vs CV; métricas completas (Precision, Recall,
 ↪  F1 macro y ponderado, ROC-AUC, PR-AUC, Brier); reliability diagrams; MDI vs
 ↪  Permutation Importance en test con la explicación del sesgo de MDI.
- **Fase 4:** DAG en DOT/networkx con T, Y, Z, M (mediador) y C (colisionador)
 ↪  justificados arco por arco; demostración del Backdoor; pregunta trampa del
 ↪  mediador y del colisionador **demostrada empíricamente** (estimar con y sin
 ↪  M, con y sin C, y mostrar el sesgo); DoWhy en 4 pasos con ATE por
 ↪  `backdoor.linear_regression` y `backdoor.propensity_score_weighting` (con
 ↪  chequeo de overlap/positividad y pesos recortados si hace falta); comparación
 ↪  contra el coeficiente ingenuo Y~T, cuantificando el sesgo de confusión;
 ↪  EconML LinearDML y/o CausalForestDML con nuisances RF, CATE con curvas por
 ↪  subgrupo e IC; las 3 refutaciones (random common cause, placebo, data subset
 ↪  80%) con interpretación; ensayo crítico "Predictivo vs Causal".
  - Si nuestro T es `random_bool` (orden aleatorio vs ranking del algoritmo de
 ↪  Expedia), aprovechalo: es una asignación aleatorizada que sirve como
 ↪  benchmark experimental para comparar contra la estimación observacional.
 ↪  Revisá además que `position` esté tratada como mediador y no como confusor, y
 ↪  que `click_bool` no se use como condición si funciona como colisionador o
 ↪  mediador de `booking_bool`. Verificá todo esto contra lo que ya está en
 ↪  `docs/dag_borrador.md`.
- **Fase 5:** roles Scrum documentados en el README y en la defensa; Quality Gate
 ↪  (pytest + ruff + chequeo de leakage en CI de `.github/`); veredicto QA
 ↪  `[APROBADO]` en cada PR; `BITACORA.md` con las 5 entradas obligatorias
 ↪  firmadas por rol (la Entrada 3 con ≥2 experimentos fallidos reales y la
 ↪  Entrada 5 con uso de IA, alucinaciones detectadas y tests que las refutan);
 ↪  Web App con los 3 módulos (Diagnóstico + PCA 2D, Predictivo "What-Is" con
 ↪  probabilidad calibrada y explicación del caso, Causal "What-If" con DAG, ATE,
```

```
- **Rúbrica complementaria:** recorré cada uno de los 15 subcriterios (1.1 a 4.4)
 ↪  y escribí qué haría falta para el nivel Excelente. Por ejemplo: análisis de
 ↪  potencia (1−β) y justificación de α (1.4); formulación matemática y DAG antes
 ↪  del código (2.1); parámetros poblacionales vs estimadores bien notados (2.2);
 ↪  modelos de resguardo como Welch, Kruskal, HC3 o umbral por costos asimétricos
 ↪  (2.3); sensibilidad propia no pedida, como con/sin outliers, distintos
 ↪  umbrales τ o subsegmentos (3.1); estabilidad con k-fold e IC bootstrap del ATE
 ↪  (3.2); gráficos compuestos de nivel publicación (3.4); informe Quarto
 ↪  independiente con resumen, índice y figuras/tablas numeradas (4.1);
 ↪  recomendaciones de negocio **cuantificadas** (4.3); reproducibilidad total +
 ↪  bibliografía formal + declaración de gobernanza de IA (4.4).

Después de la auditoría armá la lista de tareas con TaskCreate, ordenada por
 ↪  puntos en
juego (Causal 25 > ML 20 > Inferencia 15 = Gauss-Markov/PCA 15 = Deploy 15 > Scrum
 ↪  10),
y ejecutala.

---

## 2. Reglas de calidad innegociables

1. **Cero números inventados.** Toda cifra del informe, las slides, el README y la
 ↪  app tiene que salir de código ejecutado. En Quarto usá valores inline
 ↪  calculados o leídos de un `results/*.json` que genere el pipeline. Si una
 ↪  cifra del texto no coincide con la salida, el código manda.
2. **Reproducibilidad:** semilla global fija y documentada, dependencias fijadas,
 ↪  rutas relativas, script o instrucciones para bajar el dataset de Kaggle (los
 ↪  datos crudos no van en git si son pesados; si se usa una submuestra, que sea
 ↪  determinística y justificada).
3. **Notebook maestro** `notebooks/TP_Integrador_Final.ipynb` (nombre exigido por
 ↪  el enunciado), ordenado en las 4 fases técnicas, con teoría antes de cada
 ↪  bloque de código, gráficos con título y ejes, y salidas guardadas. Tiene que
 ↪  correr de punta a punta con Restart & Run All sin errores, sin celdas vacías y
 ↪  sin warnings ruidosos ni prints de debug. La lógica pesada va en `src/tpi/` y
 ↪  el notebook la importa.
4. **Sin data leakage:** el escalado, el PCA, la imputación y la selección se
 ↪  ajustan solo en train (con Pipelines). Tiene que haber tests en pytest que lo
 ↪  verifiquen.
5. **Lenguaje estadístico preciso:** "no se rechaza H0" (nunca "se acepta");
 ↪  distinguir parámetro (β, μ, τ) de estimador (β̂); reportar estadístico, gl y p
 ↪  en cada test.
6. **Redacción:** español rioplatense técnico, sobrio y denso, en primera persona
 ↪  del plural ("estimamos", "decidimos"), con tono natural de estudiantes. Nada
 ↪  de relleno ni frases que suenen a IA. Ecuaciones en LaTeX.
7. **Verificación final obligatoria:** `pytest` en verde, `ruff check` limpio,
 ↪  notebook ejecutado completo, `quarto render` del informe y de las slides sin
 ↪  errores. Abrí los PDF y HTML generados y revisalos: figuras que se vean,
 ↪  numeración, índice, referencias resueltas, sin `??` ni placeholders tipo
 ↪  TODO/XXX/lorem.

---
```

```
## 3. Git y GitHub

- Integrá el trabajo de las ramas existentes y del PR abierto (revisá los
 ↪  conflictos con criterio, no sobrescribas a ciegas).
- Trabajá en ramas por fase y abrí PRs con el checklist del Quality Gate. Antes
 ↪  del merge a `main`, dejá en cada PR un comentario de QA con el veredicto
 ↪  `[APROBADO]` o `[BLOQUEADO]` y su evidencia (tests, ruff, leakage). Esto lo
 ↪  exige el enunciado.
- Cerrá o actualizá los 37 issues según corresponda, con referencia al commit o
 ↪  PR, y agrupalos en milestones por fase.
- Commits convencionales en español (`feat(fase-4): ...`). **Nunca** hagas force
 ↪  push a `main` ni borres ramas sin preguntarme.

---

## 4. Deploy de la Web App

- La app (`app/`, Streamlit) carga modelos y resultados serializados
 ↪  (joblib/json); no reentrena al iniciar. Tiene que levantar rápido y con datos
 ↪  livianos (una muestra incluida en el repo, si hace falta).
- Probala localmente (`streamlit run`) y verificá los 3 módulos.
- Dejá todo listo para Streamlit Community Cloud (requirements de la app, versión
 ↪  de Python, `.streamlit/config.toml`) y además un `Dockerfile` testeado como
 ↪  plan B.
- El deploy en la nube requiere mi login. Cuando llegues ahí, dame los pasos
 ↪  exactos, cortos y numerados para hacerlo yo. Cuando te pase la URL, agregala
 ↪  al README, al informe y a las slides.

---

## 5. Informe, slides, README y anexo de IA

- **`report/informe.qmd` → `informe.pdf` + `informe.html`:** portada (UCA,
 ↪  materia, profesor Ivan Gonzalo Vilaboa, integrantes con su rol, fecha),
 ↪  resumen ejecutivo, índice, introducción y preguntas de negocio, datos y
 ↪  auditoría de calidad (antes/después de la limpieza), las 4 fases, el ensayo
 ↪  crítico, **recomendaciones de negocio cuantificadas** (por ejemplo, lift
 ↪  causal en conversión o reservas y a qué segmentos conviene aplicar el
 ↪  algoritmo según el CATE), limitaciones y amenazas a la validez, conclusiones,
 ↪  bibliografía formal (`referencias.bib`: Pearl, Chernozhukov et al. DML, docs
 ↪  de DoWhy y EconML, Kaggle, etc.) y anexos.
- **Anexo "Uso de IA" (obligatorio):** herramientas usadas (Claude / Claude Code),
 ↪  resumen de las conversaciones y de los prompts de diseño principales, cómo se
 ↪  trabajó con el agente (harness, tareas delegadas), alucinaciones y errores
 ↪  detectados, tests determinísticos que los refutan, y una declaración de
 ↪  alcance y auditoría humana. Basalo en `docs/ia_log.md` y alineado con la
 ↪  Entrada 5 de la bitácora. Incluí este mismo prompt como ejemplo.
- **`report/slides.qmd` → `slides.pdf`:** de 10 a 12 slides siguiendo la
 ↪  estructura recomendada del enunciado (problema →
 ↪  inferencia/ANOVA/Gauss-Markov/PCA → torneo ML y calibración →
 ↪  DAG/ATE/CATE/refutaciones → demo de la app → conclusiones y bitácora), para 12
 ↪  a 15 minutos.
```

```
- **`docs/defensa.md`:** guion de la exposición y banco de preguntas probables del
 ↪  tribunal, con respuestas cortas por rol (sobre todo: por qué Shapiro no sirve
 ↪  con N grande, la trampa del mediador y del colisionador, por qué MDI está
 ↪  sesgado, qué hace DML, qué significa cada refutación).
- **`README.md`:** de calidad editorial, con badges (CI, Python, deploy),
 ↪  diagrama del pipeline y del DAG, estructura del repo, roles, reproducción paso
 ↪  a paso, link a la app, al informe y a las slides.

---

## 6. Al terminar, entregame esto

1. **Lista de entregables finales** con su ruta exacta, para mandarlos (informe
 ↪  PDF/HTML, notebook ejecutado y su export HTML, slides PDF, BITACORA.md, link
 ↪  del repo, link de la app). Si conviene, armá un `entrega/` o un .zip con todo
 ↪  junto.
2. **Tabla final de la rúbrica:** cada subcriterio, el nivel que estimás que
 ↪  alcanza y la evidencia.
3. **Lo que tengo que hacer yo a mano** (deploy, revisar algo puntual, subir a
 ↪  EVA), en pasos cortos.
4. **Riesgos residuales**, si los hay, explicados con honestidad.

No me digas que algo está terminado si no lo ejecutaste y verificaste. Si
 ↪  encontrás un
error metodológico en lo que ya hicimos, corregilo y anotalo en la bitácora; no lo
escondas.
```

13. Anexo B: reproducibilidad y Quality Gate

- **Entorno.** Python 3.11; dependencias fijadas en `requirements.txt` (con el motivo de cada versión fijada) y `app/requirements.txt`; semilla global `SEED = 42` (`src/tpi/config.py`).
- **Datos.** `python data/download_data.py` descarga `data.zip` de Kaggle (requiere aceptar las reglas de la competencia), verifica hashes contra `data/manifiesto.json` y genera la muestra determinística. Ningún dato se versiona (`tests/test_confidencialidad.py`).
- **Ejecución.** `python -m nbconvert --to notebook --execute --inplace notebooks/TP_Integrador_Final.ipynb` regenera figuras, `models/resultados.json` y los artefactos de la app; `quarto render report/informe.qmd` regenera este informe.
- **Quality Gate (CI en cada PR):** `ruff check` y `ruff format --check` (PEP 8), tests de no-fuga (`pytest -m leakage`), la suite completa de `pytest` y la construcción de la imagen Docker de la app. El veredicto de QA (`[APROBADO]` o `[BLOQUEADO]`) se deja en cada PR antes del merge a `main`.
- **App.** `streamlit run app/app.py` o `docker build -t tpi-eaymr-app . && docker run -p 8501:8501 tpi-eaymr-app`; el enlace público del deploy figura en el README del repositorio.